(Dead-man-switch)

Permission-Based Wallet

A self-custodial Cardano wallet with shared permissions, spending limits, scheduled payments, and key-loss recovery

Shipped as **Epora**

Whitepaper

Project Catalyst Fund 11 — Cardano Use Cases: Concept

Author	Sandro Schaier
Platform	Cardano (Plutus v3 / Aiken)
Version	1.0 (June 2026)

Open-source. Built for the Cardano community.

Abstract

Wider adoption of Web3 and DeFi is held back by the lack of a reliable way to manage permissions. Traditional wallets can be backed up, seed phrases stored in vaults, extra hardware wallets provisioned, and access arranged for family members once they learn to use Web3; but each of these measures adds its own complexity and has its distinct problems.

Backups can be lost and require safe storage, hardware wallets risk breaking over time, and giving access to family members means trusting not only them but also their security measures.

Smart-contract and multi-signature wallets exist, for example for projects that manage large treasuries. While these tools spread risk and reduce the attack surface, they were built for large organizations and dedicated use-cases: the result is custom contracts, often not public, that favor function over usability. Ordinary users however have the same needs and few options that fit them.

This wallet, released as *Epora*, brings that treasury-management approach to individual users. Rather than ship a dedicated contract per feature and requirement, it threads the whole configuration through a single state-thread token (STT) datum and bounds every movement of funds with a two-validator handshake, so features compose as configuration on one reviewable contract that can be customized to the user's needs. Because funding works through ordinary transactions, it preserves interoperability with existing wallet infrastructure and a familiar dApp-like user experience.

The contracts and interface are open source; the implementation runs on the Cardano network.

Contents

Abbreviations

API Application Programming Interface

STT State-Thread Token

UTXO Unspent Transaction Output

Glossary

Wallet A traditional wallet is a program that stores a secret key and signs transactions to interact with a blockchain. Hardware wallets are a variation that keeps the secret on a dedicated device

Beneficiary In this context a beneficiary is any other person, represented by a traditional wallet, who may receive funds once the proof-of-life window (and any personal delay) has lapsed

Owner In this context an owner is a person with admin or sufficient agreeing multi-signature users with power over the wallet. By default the wallet has a single admin owner with broad access to the funds; this access can be restricted, for example by requiring a multi-signature quorum across several owners

Smart Contract Software whose execution is validated by a decentralized network, so its results can be trusted; deployed on a blockchain to offer on-chain functionality

Validator The EUTXO counterpart of a smart contract: a script attached to an output that runs when the output is spent and accepts or rejects the whole transaction

dApp A decentralized application: software, typically a web app, through which users interact with a blockchain and its smart contracts

Vesting Locking funds until a chosen date, before which the beneficiary (or owner) cannot access them

State Machine A computer-science model that describes a system as a set of states and the transitions between them; used here to model the wallet's reachable states

Proof of Life Proof of Life is the dead-man-switch heartbeat: a renewal of the wallet's unlock-time deadline. It is satisfied by any operator action that renews the window, or by a dedicated liveness-keeper renewal transaction through the dApp. If it is not provided before the deadline, beneficiaries may unlock

State-Thread Token (STT) A single native token, unique per wallet, whose continuing UTXO carries the wallet's authoritative configuration (the State) as its datum. Exactly one exists per wallet and it must be forwarded on every transition, so it is always the single source of truth for that wallet's rules

Streaming Payment A recurring payout that accrues linearly between a start and end date to a fixed payee. The accrued-but-unpaid amount may be settled by anyone (a permission-less crank), since the contract fixes the destination and caps the amount at what has accrued, but also guarantees accrued funds always remain accessible to the payee and cannot be diverted by anyone

1 Introduction

Self-custody is the core promise of a public blockchain: the holder, and only the holder, controls the funds. That same property is its sharpest edge. A lost seed phrase, a hardware failure, a sudden death, or an incapacitating accident can put funds permanently out of reach, with no path to recovery. The very absence of an intermediary that makes self-custody trustless also removes the intermediary that, in traditional finance, handles inheritance, power of attorney, joint accounts, spending limits and other needs of regular users.

Existing answers are partial. A hardware wallet protects a key but does nothing if that key is lost or its holder is gone. Conventional multi-signature distributes trust but has no notion of time or inactivity, and remains operationally heavy. Custodial services restore familiar recovery only by taking custody away. What is missing is a wallet that keeps custody self-sovereign while encoding, on-chain, the everyday safety nets people expect: spending limits, joint control, automatic recurring payments, multi-wallet backup and a way for trusted parties to recover funds if the owner goes silent, with none of those parties able to seize funds while the owner is active.

This paper presents a permission-based (dead-man-switch) wallet for Cardano, funded under Project Catalyst [1, 2] and released to users as *Epora*. A single shared wallet is governed by an explicit, on-chain permission model rather than by one all-powerful key. An owner retains broad control while active. Beneficiary parties may recover funds only after the owner goes silent. Spenders may hold capped daily budgets, and recurring payouts settle to fixed recipients automatically. The whole configuration lives in one datum, carried by an STT, and every movement of funds is bounded by a two-validator handshake.

Contributions. This paper makes three contributions:

- A *features-as-configuration* architecture (Section 4) that replaces a combinatorial family of bespoke contracts with one reviewable contract whose behavior is selected by configuration. We treat that simplicity not only as a UX benefit but also as a security decision.
- A *permission and recovery model* (Section 5) that combines weighted-share, multi-beneficiary recovery, a proof-of-life dead-man-switch, per-day spending allowances, and streaming payments any stakeholder may settle. The closest prior art (Section 10) has no such combination.
- An asset-based *security analysis* (Section 7) that states the threat model first, lists the protocol's assets, and gives the invariant defended for each; the design's limitations and trust assumptions are collected in Section 8. The model and these invariants are restated formally, with proof sketches, in Section 6.

2 Background: The Extended UTXO Model

Cardano uses the Extended UTXO (EUTXO) model [3], which underlies most design decisions in this work and is worth contrasting with the account model that many comparable wallets target.

In the account model popularized by Ethereum, global state is a set of accounts. Each is identified by a 20-byte address and holds a nonce, a balance, and, for contract accounts, code and persistent storage; a contract mutates its storage in place as transactions arrive [4]. In the UTXO model, there is no mutable per-contract storage: state is the set of unspent transaction outputs, and a transaction consumes some outputs and creates others. EUTXO extends each output with an arbitrary *datum* and guards spending with a *validator* script. That script sees the whole transaction (its inputs, outputs, value, signatures, and validity interval) and returns only accept or reject.

Two consequences matter here. First, EUTXO validation is deterministic and local: a script's verdict depends only on the transaction being validated, so the cost and outcome of a spend are known before submission and cannot be changed by unrelated on-chain activity. This suits a permission wallet, whose rules read directly off explicit inputs, outputs, signers, and time bounds. Second, with no in-place mutable storage, a wallet that needs persistent, evolving configuration must *thread* that state through a chain of outputs. The usual way to do this is a *state-thread token* (STT): a unique token whose continuing output always carries the current state as its datum, so the wallet's state is just the one output holding that token. However, this usually requires specialized transactions that encode a datum for every UTXO, which would break compatibility with existing UX flows. Therefore, while we adopt this pattern, described in Section 4, we also introduce a two-validator handshake to keep compatibility with existing wallet flows.

3 Design Goals and Threat Model

3.1 Design goals

G1 — Self-custody preserved.
No party may move funds outside the rules the owner configured; there is no privileged or centralized recovery, administrator override, or upgrade key over user funds.

G2 — Recovery is reachable.
A configuration that could permanently strand funds with no way back in must be rejected before it is ever created.

G3 — Authorization over restriction.
Safety derives from *who* may act and *when* (dead-man-switch timing), not from attempting to constrain where an authorized actor may send funds. This keeps the trusted path simple and avoids brittle policy-stacking.

G4 — Simplicity as a security property.
Unnecessary complexity is a leading root cause of vulnerabilities, and limiting complex logic is itself a decision that enables open external review [5]. The design therefore favors one small, auditable contract over a large or generated family of them.

G5 — Liveness without custody.
Recurring obligations must continue to settle even when the owner is offline or gone, without granting anyone custody. Streaming payments must stay funded, and funds accrued to third parties must remain withdrawable.

3.2 Threat model

A security claim is only meaningful against a stated adversary [5]. We assume an adversary who can submit arbitrary transactions, observe all on-chain data, choose transaction validity intervals freely within ledger rules, and supply their own inputs and outputs. The adversary may act as a permission-less third party, or control *some* keys (a single spender, a single beneficiary, a minority of multi-signature power), but not a quorum of owners. Cryptographic primitives and the Cardano ledger are assumed sound. Off-chain key custody at the edges (how a user stores their own key) is out of scope, as is transaction-level privacy. However, we provide ways to minimize these risks.

We do not apply a generic checklist such as STRIDE, which was built for software with few, mutually trusting parties and is weak at surfacing financial and collusion threats in cryptocurrency systems [6]. Instead we follow an asset-based method [6]: list the protocol's assets, state the secure behavior (the invariant) for each, and treat any violation as a threat. Section 7 carries out that analysis against the assets in Table 1.

Asset	**Invariant to defend**
Wallet funds (locked UTXOs)	Move only as a validated action permits
STT / State datum	Exactly one STT; state changes only as declared
Proof-of-life deadline	Each renewal advances it by at most one increment
Allowance counters	Spend bounded; daily reset cannot be forged
Streaming-payment reserve	Accrued-but-unpaid amounts cannot be diverted
Beneficiary share	At most a weighted share; one withdrawal each
Stake rights (rewards, delegation, vote)	Rest only at the intended stake credential
Wallet operability	State fits the execution budget; a way in always remains

Table 1: Protocol assets and the invariant defended for each.

4 System Architecture

4.1 Features as configuration

A permission wallet must support many features (broad owner control, daily allowances, multi-signature, time locks, a dead-man-switch, streaming payments) and, above all, their *combinations*. Even with six features the number of distinct "and"/"or" combinations is

$$\sum_{k=1}^{6} \binom{6}{k} \times 2^{k-1} = 364,$$

and the growth is exponential: in general $\sum_{k=1}^{n} \binom{n}{k} 2^{k-1} = (3^n - 1)/2$, so a seventh feature already yields 1093. Building, testing, and auditing one customized contract per combination is therefore infeasible, and generating contracts on demand only moves that complexity, and its review burden, onto the user.

We take the opposite route. The wallet uses *one* contract whose features are *configuration*. The complete configuration (owners and their multi-signature threshold, beneficiaries, the proof-of-life timer, and the streaming payments) lives in a single *State* datum carried by the STT (Figure 1). Each feature is a field of, or an action on, that one object, so combinations are data a user selects rather than new code to audit. Under goal G4, collapsing the design to a small, auditable contract is a security and reviewability decision: it is what makes a full, open review tractable [5].

State	the STT datum: the entire wallet configuration
access.users[]	owners, spenders, liveness keepers: weight, per-day allowance (≤ 15)
access.multi_sig_threshold	θ, the weighted quorum for operator actions
access.beneficiaries[]	weight ≥ 1, optional unlock delay (≤ 25)
proof_of_life	unlock deadline d and renewal increment Δ
streaming_payments[]	payee, rate, start/end, paid-out (≤ 25)
wallet_name	optional human label ($\leq$ 32 bytes; admin-only rename)
intended_stake_credential	the stake credential every wallet output must carry
last_non_admin_payout_at	shared stamp of the latest non-admin payout or payee cancel; anchors their cadence limit

Figure 1: The State datum carried by the STT. Every feature is a field, so feature combinations are data the owner selects, not new code to audit.

4.2 State-thread token and the two-validator handshake

The STT pattern, a multivalidator that both mints a unique state token (a Cardano native asset [7]) and governs every spend of the output carrying it, is established EUTXO practice [8, 9]; we do not claim it as novel. What this design adds is a deliberate *split* of enforcement across two validators that share one redeemer (the declared action) as their single source of truth (Figure 2):

- The **STT validator** authorizes every state transition and proves the declared action equals the true change to the State. For example, it checks that a declared allowance spend equals the actual decrease in the user's remaining allowance.
- The **wallet validator** is bound to the specific state-thread token and holds the funds. During spends it bounds how much value may leave, checking movement against the *same* declared action the STT validator just proved honest.

Guarantee

The **STT validator** proves *declared action = true state change*; the **wallet validator** proves *movement $\leq$ declared action*. Composed, **fund movement out of the wallet is bounded by the true change to the wallet's configuration**: no actor can move more than the

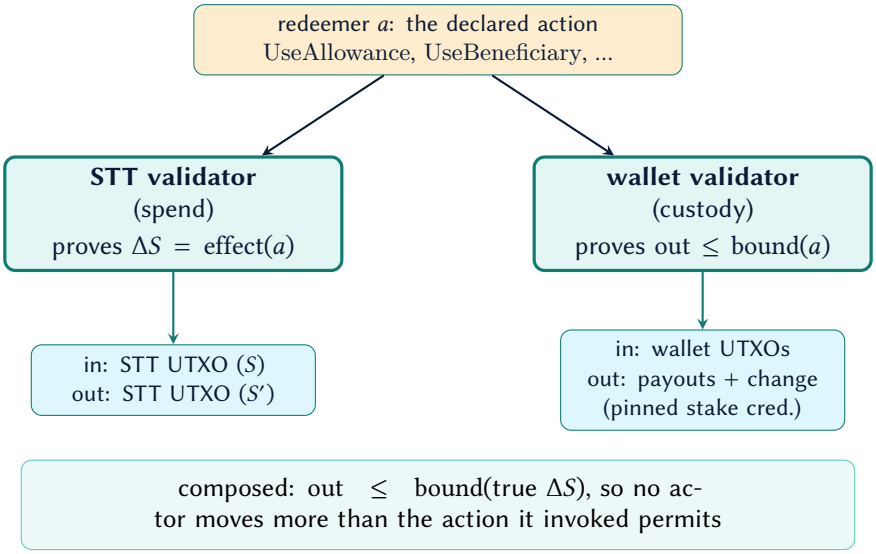

Figure 2: The two-validator handshake. Both validators read the *same* redeemer: the STT validator ties the declared action to the true state change, and the wallet validator bounds outgoing value by that action.

> action they invoked permits.

Because the two validators agree on one redeemer, there is no second, parallel encoding of intent to keep in sync, which is itself a simplicity gain (G4). For cost optimization the contract code stays in a permanent (spend always-fail) smart contract as a reference script, hosting the STT's script at a known location, so spends stay small without trusting a mutable pointer.

4.3 Why two contracts

Splitting the system into a *configuration* contract (the STT validator) and a *custody* contract (the wallet validator) separates two concerns that change at different rates and carry different risk. The wallet contract holds the funds but contains no permission logic of its own, while the STT contract reasons about permissions but never directly guards the bulk of the funds. Each can be read and audited on its own, and the funds live behind a small validator whose entire job is a value bound.

This separation also yields the property that makes funding the wallet straightforward. Because the custody contract places no conditions on what it receives, its address behaves, on the receiving side, like any ordinary Cardano address. A deposit is therefore an ordinary payment, and any existing payor can fund the wallet without touching the STT UTXO. This also allows funds to be spread across parallel UTXOs, easing the size restrictions the ledger places on a single UTXO. The next subsection defines the mechanism precisely.

4.4 Receiving funds needs no datum

In the EUTXO model, a validator script runs only when an output it guards is *spent*, never when an output is *created* [3, 10]. There is no restriction on the outputs sent *to* a script address: anyone may pay a plain output, with no datum and no special construction, to the hash that identifies the wallet's custody contract, exactly as they would pay an ordinary key address [10]. The wallet's validator does not need to "accept" the deposit, because it is not consulted on the way in; it is consulted only later, when those funds are spent and the permission rules apply. This has various advantages; above all, it removes a classic failure mode of script wallets whose deposits must carry a datum, where one misconfigured send with a missing or invalid datum bricks the funds: here there is no datum to get wrong.

Guarantee

A permission wallet's address behaves, for the purpose of receiving, like any ordinary Cardano address. To fund it, a sender pays to the wallet's script address with no datum, no redeemer, and no awareness that a smart contract is involved at all. The permission rules engage only when funds leave.

For users, funding stays simple: a wallet can publish a single receive address and accept payments from any Cardano wallet, exchange, payroll system, or dApp, without those senders attaching a datum, integrating a library, or even knowing the destination is a smart contract. Shared treasuries, streaming-payment funding, and ordinary top-ups all work through this one address. The complexity of the permission model is paid for only by those who *spend*, never by those who *pay in*.

4.5 Pinning the stake credential

A Shelley address carries two independent parts: a *payment* credential that governs spending, and a *stake* credential that governs delegation and staking rewards [11]. The two are freely combinable: any payment credential may be paired with any stake credential. Combined with unrestricted receiving (Section 4.4), this opens a subtle gap. Anyone may pay to the wallet's payment credential under a stake credential *they* control, a so-called "Frankenstein" address. Spending such an output still requires the wallet validator, so the funds cannot be stolen. But the foreign stake credential would collect their staking rewards and direct their delegation and governance vote, and an ordinary address-based balance query would not even see them. The same drift can arise innocently, from a deposit made to an outdated address.

The State therefore records an *intended stake credential*, and the wallet validator requires every continuing wallet output to carry exactly it. Therefore no spend can re-home funds to a different stake credential, and the intended stake credential can eventually be reached for all UTXOs. Wallet *inputs* can still be aggregated by payment credential (the same accounting that defends the credential-aggregation invariant in Section 7), so funds that arrived at any other stake credential can always be swept back to the intended one by a value-preserving consolidation. The intended

credential is changed only by an admin or a multi-signature quorum, through a dedicated transition, so re-delegating the wallet is a deliberate, narrowly authorized act rather than a side effect of any spend.

Guarantee

Every unit of wallet value that remains in the wallet after a spend comes to rest at the wallet's *intended* stake credential. Staking rewards, delegation, and governance weight on wallet funds thus stay under the wallet's control, changeable only by an authorized operator (admin or multi-signature quorum) through a dedicated transition.

5 Permission and Recovery Model

The wallet's complete behavior is a small, fixed vocabulary of transitions over the State. Table 2 summarizes, for each transition, who may invoke it and how much value may leave the wallet; the subsections that follow describe the security-relevant mechanics.

5.1 Owners, allowances, and multi-signature

An owner authorized through the admin or multi-signature path may move wallet value broadly (goal G3); the only floor is the streaming-payment reserve (Section 7). Multi-signature is *weighted*: each owner carries a signing power, and an action is authorized only when the summed power of the signing owners meets a configured threshold (for example, weight 3 of a total 5). A spender, by contrast, holds a per-day allowance and may draw only up to it; the allowance resets on a rolling deadline anchored to the transaction's *earliest* validity time, so a user cannot forge an "already reset" view by stretching the validity window.

5.2 The dead-man-switch

The proof-of-life timer drives the recovery model. While the owner provides proof of life before each deadline, whether by spending or by a dedicated liveness-keeper renewal, the wallet is "alive" and beneficiaries cannot act. A single renewal may push the deadline forward by at most one configured increment, so the window cannot be skipped arbitrarily far ahead in one transaction. If the owner goes silent and the deadline passes, recovery shifts from owners to beneficiaries. Figure 3 shows the reachable states; Figure 4 traces the same mechanics along a timeline.

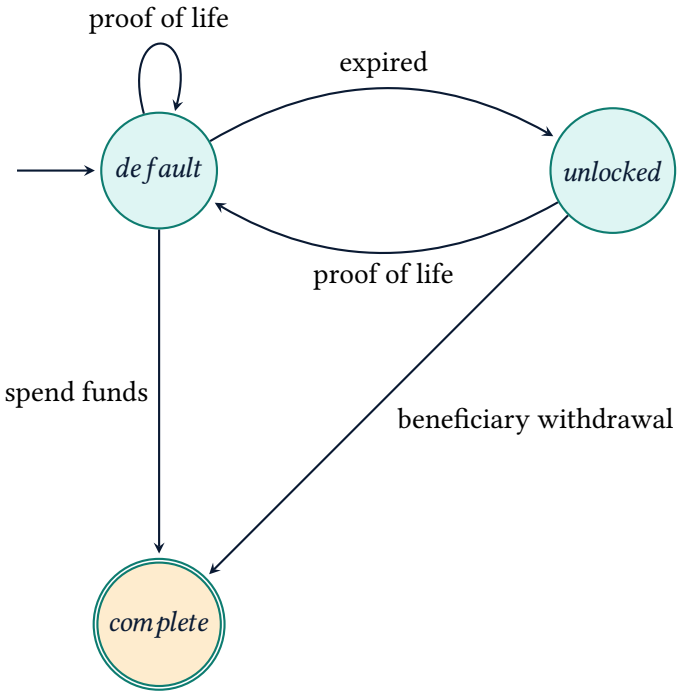

Figure 3: The dead-man-switch state machine: states reachable by the proof-of-life timer.

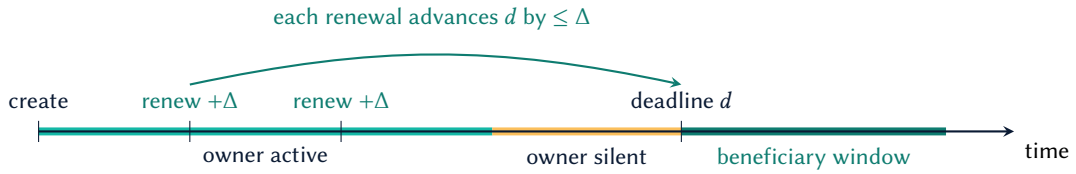

Figure 4: Proof-of-life timeline. While the owner renews, each renewal advances the deadline d by (usually) Δ and beneficiaries cannot act. Once the deadline passes with no renewal, each beneficiary may withdraw, after any personal delay of its own (e.g. a 21st birthday).

5.3 Weighted-share beneficiary recovery

Once the timer (and a beneficiary's own optional delay) has lapsed, a beneficiary may withdraw at most its *weighted share* of the distributable pool: $\text{weight}/(\Sigma\,\text{weights}) \times (\text{funds} - \text{reserve})$ per asset. The acting beneficiary is then removed from the State in the same transition. Because each weight is retired on use, the remaining beneficiaries' shares re-normalize against the reduced total, and the set collectively recovers the entire distributable balance whatever the withdrawal order. No beneficiary can take more than its share, and none can withdraw twice.

5.4 Streaming payments and open settlement

A streaming payment (surfaced in the product as a scheduled payment) accrues linearly between a start and end date to a fixed payee. Whatever has accrued but not yet been paid out may be settled by any *stakeholder* in the wallet — an operator, any listed user, the payee itself, or an unlocked beneficiary — because the contract fixes the destination and ties the value moved to the settlement recorded in the State, capped at the accrued, unpaid amount; safety is structural, not identity-based (goal G5). This decouples a recipient's income from the owner's liveness, and guarantees the paid funds remain locked to the recipient, with full self-custody over them, which matters not only in the dead-man-switch case: a salary or subscription keeps flowing, until stopped, even if the owner is offline or gone, and accrued funds always stay available to the recipient, even if the owner or beneficiaries try to withdraw them. This means the amount owed to active streaming payments forms a *reserve* that no other spend, not even an owner's, may draw below.

Settlements consume and re-create the single STT output, so a party able to crank at will could occupy the state thread and crowd out the owner's own transactions, including a proof-of-life renewal, at only fee cost. Two guards close this: *who* may settle, and *how often*.

Settlement is open but not anonymous. It requires a signature from a party with a stake in this wallet: an admin, a multi-signature quorum, any listed user, the payee of any stream in the set, or an unlocked beneficiary. The payee arm is what keeps income reachable when every wallet key is lost — the party actually owed the money can always pull it — so openness is preserved where it matters without leaving the state thread exposed to arbitrary third parties.

Cadence is then bounded for everyone but the admin. The State stamps each non-admin payout's upper validity bound, and the next cadence-limited streaming action is accepted only when its earliest validity time lies at least thirty minutes past that stamp, with the validity window itself capped at one hour. Payee cancellation shares this same clock: a cancellation delays the next non-admin payout and a payout delays the next cancellation. Only an admin payout bypasses the limit, and it leaves the stamp untouched. Throttling recovery is acceptable because a single settlement can clear the stream set in one transaction, so a recovering beneficiary never needs a burst. Section 6 states the cadence this yields.

An earlier revision of this design made settlement fully permission-less and relied instead on a *progress* requirement — the streaming-payment set had to change — to stop no-op churn. That requirement was ineffective: one unit of progress satisfies it, so the churn it was written against remained available at fee cost. It has been removed, and the authority gate above replaces it.

Streaming payments are introduced at wallet creation or added afterwards by an operator (an admin or a multi-signature quorum) through the manage path; a newly added payment is always born unsettled, so it cannot fabricate already-paid progress. The other later changes are to settle accrued value (the crank, which also removes a payment once it has matured and been fully paid out) or, for an operator, to reschedule a payment's end date, moving it later to extend the payment or earlier to stop further accrual (never below the transaction's upper validity bound, so already-accrued value cannot be clawed back); an operator stops a payment by rescheduling its end date down, not by deleting the entry, so an existing payee can never be dropped. Existing payees are therefore untouchable except for this no-clawback reschedule, while the reserve accounting stays sound across additions because the per-asset reserve is recomputed from the live State's payee set

on every spend rather than assumed fixed; the count cap (Section 6) bounds the set however it grew.

A payment's own payee may also shorten it through a dedicated cancel path authorized by the payee's signature rather than operator authority. The new end must be strictly earlier than the old end but no earlier than both the payment's start and the transaction's upper validity bound, the ledger-safe proxy for "now". If the payment has not started, this permits the end to land exactly on its start, creating a zero-lifetime obligation without even one millisecond of forced accrual. Such an entry owes and reserves zero and must be removed, rather than retained, by the next payout; fresh schedules remain strict positive-duration, zero-paid entries. The action cannot extend the payment, claw back value already accrued, touch another payment, or move wallet value. It shares the global non-admin streaming cadence: the transaction must use a validity window of at most one hour, must begin at least thirty minutes after the previous stamp, and writes its own upper bound as the next stamp. There is deliberately no persistent cancellation flag. The payee may shorten the same payment again after the cooldown, and an admin or multi-signature quorum may later reschedule it; this bounded, signature-authorized, fee-funded repetition is an accepted liveness tradeoff. A payee whose payout address is a script credential cannot produce a signature and so has no self-cancel path; such a payment is managed by an operator.

6 Formal Model

This section makes the model of Section 5 precise: the state space, the transitions over it, the bound each transition places on outgoing value, and the invariants the two validators enforce, stated as theorems with proof sketches. The definitions mirror the Aiken types and on-chain checks, so each proof reduces to the predicate the contract evaluates. A transaction fixes a validity interval $[t_0, t_1]$ (its earliest and latest admissible times), and $D = 86\,400\,000$ is the number of milliseconds in a day.

6.1 State and notation

Definition 1 (Wallet state)**.** A wallet state is a tuple $S = (U, \theta, B, P, M, sc, \tau)$: a list U of user records, a weighted multi-signature threshold θ (possibly unset), a list B of beneficiaries, a proof-of-life setting $P = (d, \Delta)$ pairing the unlock deadline d with the renewal increment Δ (set together, or absent when the dead-man-switch is unused), a list M of streaming payments, the intended stake credential sc, and a cadence stamp τ, the upper validity bound of the latest cadence-limited payout or payee cancellation (absent until the first). Each user u carries an admin flag, a renewal flag, an optional signing power, a per-day allowance perDay_u, a remaining allowance rem_u, and a reset deadline ρ_u; each beneficiary b carries a weight $w_b \geq 1$ and an optional personal delay. A streaming payment carries no cancellation marker: its end date alone determines when accrual stops, and equal start and end dates represent a zero-lifetime obligation. Every key or script credential hash stored in State is exactly 28 bytes, matching the ledger's Blake2b-224 credential width; this is checked at mint and every State ingress path rather than inferred from Aiken's byte-array aliases. Pointer stake addresses are rejected at the same boundary because new pointer addresses are unavailable in the deployed Conway-era protocol; payout addresses are therefore enterprise

addresses or use an inline, width-checked stake credential. Asset identifiers are canonical too: ada is represented only by an empty policy and empty name, while a native asset has an exactly 28-byte policy id and an asset name of at most 32 bytes. The lists are bounded: $|U| \le 15$, $|B| \le 25$, $|M| \le 25$.

6.2 Streaming accrual and reserve

Definition 2 (Accrual)**.** For a streaming payment p with per-day rate r_p, start s_p, and end e_p, the amount accrued by time t is

$$\mathrm{accr}_p(t) \;=\; \left\lfloor \frac{\big(\min(t,\, e_p) - s_p\big)\, r_p}{D} \right\rfloor .$$

Definition 3 (Per-asset reserve)**.** The value the wallet must retain for an asset x at time t is the accrued-but-unpaid total over the payments denominated in x, where paid_p is the amount already settled to p's payee, with a one-unit rounding buffer each:

$$R_x(t) \;=\; \sum_{p \in M_x} \max\big(0,\; \mathrm{accr}_p(t) + 1 - \mathrm{paid}_p\big), \qquad M_x = \{\, p \in M \;:\; \mathrm{asset}(p) = x \,\}.$$

6.3 Transitions and bounds

Definition 4 (Transition)**.** An action a is one of

$$\begin{aligned} &\mathrm{RunOperator}(\pi, \kappa), \quad \mathrm{RenewProofOfLife}, \quad \mathrm{UseAllowance}(\delta),\\ &\mathrm{UseBeneficiary}(i), \quad \mathrm{PayStreamingPayment}(\delta), \quad \mathrm{Consolidate}(\pi),\\ &\mathrm{CancelStreamingPayment}(j), \end{aligned}$$

with authority path π (admin or a multi-signature quorum; for Consolidate also an unlocked beneficiary), declared per-asset payout δ, beneficiary index i, streaming-payment id j, and operator kind κ, one of Use, UpdateState, ManageStreamingPayments, RemoveAccessIndex, or SetIntendedStakeCredential (the five operator rows of Table 2). A transaction tx drives a transition $S \xrightarrow{a} S'$, accepted exactly when (i) the authority required by a is met, (ii) $S' = \mathrm{Post}_a(S, tx)$, and (iii) the net value leaving the wallet is at most $\mathrm{bound}_a(S, tx)$. CancelStreamingPayment(j) is authorized by the signature of payment j's payee; its Post chooses an end e'_j satisfying $\max(s_j, t_1) \le e'_j < e_j$, leaves every other payment fixed, stamps $\tau' = t_1$, and moves no value. Thus a pre-start cancel may have zero lifetime. Operator management may preserve or extend that existing zero-duration form; its usual $s_j + 1$ stop floor applies to positive-duration inputs and fresh schedules. Cancellation is governed by the same finite-window and cooldown conditions as a non-admin payout. Here Post_a is the constrained post-state the action prescribes; we write ΔS for the actual change from S to S' and $\mathrm{effect}(a)$ for the permitted change Post_a describes, so condition (ii) is equivalently $\Delta S = \mathrm{effect}(a)$, the form the STT validator checks. Table 2 gives bound_a; Figure 5 shows one transition in full. PayStreamingPayment is authorized by any *stakeholder* — an admin, a quorum, any user in U, the payee of any payment in M, or an unlocked beneficiary — and adds acceptance conditions of its own on the validity interval and cadence stamp τ, given in Theorem 7.

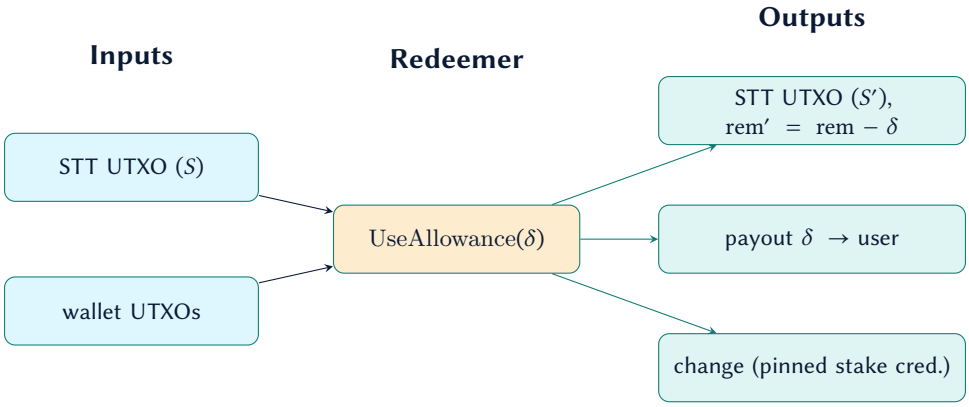

Figure 5: Anatomy of one transition, an allowance spend. The STT validator checks that the declared δ equals the drop in the user's remaining allowance; the wallet validator checks the payout is exactly δ and the change keeps the pinned stake credential.

6.4 The two validators

Enforcement is split across two scripts that read the *same* redeemer a (Section 4.3, Figure 2).

Definition 5 (STT validator). V_S accepts $S \xrightarrow{a} S'$ only if the declared action equals the true state change, $\Delta S = \text{effect}(a)$, exactly one state token is minted at creation, and exactly one continuing output carries it.

Definition 6 (Wallet validator). V_W accepts a spend only if, for every asset x, the continuing wallet value satisfies $\text{out}_x \geq \text{in}_x - \text{bound}_a(x)$, where $\text{bound}_a(x)$ is the asset-x component of $\text{bound}_a(S, tx)$, and, for every action *except* PayStreamingPayment, the reserve floor $\text{out}_x \geq \min(\text{in}_x, R_x(t_1))$. The settlement is exempt because its outflow is already pinned per asset to the tagged payee outputs, so it cannot underfund a payee; applying the floor to it would instead deadlock an underfunded wallet, since the floor forbids every outflow once $\text{in}_x \leq R_x(t_1)$ — including the only action that reduces R_x (Section 8, "Under-funded settlement"). Every continuing wallet output must additionally carry the credential sc and no reference script.

6.5 Invariants

Theorem 1 (Bounded movement). *For every reachable S and accepting transaction with action a, the net value leaving the wallet is bounded, per asset, by* $\text{bound}_a(S, tx)$*, taken against the* true *change to S.*

Proof sketch. V_S forces $\Delta S = \text{effect}(a)$, so the declared action is the real one: for UseAllowance(δ) the declared δ equals the actual drop in rem_u, and so on. V_W independently forces $\text{out} \geq \text{in} - \text{bound}_a$ for every asset. Both read the same a, so their composition bounds outgoing value by the action's true effect. □

Theorem 2 (Single state thread). *From creation onward, exactly one state token exists, and the configuration is always the datum of the unique output carrying it.*

Proof sketch. Minting admits exactly one token; every spend forces exactly one continuing output bearing it and rejects a transaction presenting a second or none. The claim follows by induction on transitions. □

Theorem 3 (Allowance velocity). *A spender cannot draw more than* perDay_u *of any asset within any half-open rolling window of length D, whatever validity interval the transaction declares.*

Proof sketch. The available allowance is perDay_u if $t_0 \geq \rho_u$ and rem_u otherwise, and the spent amount is $\delta = \text{available} - \mathrm{rem}'_u \geq 0$. The reset is anchored to the *earliest* bound t_0, while the next deadline is $\rho'_u = \max(\rho_u,\, t_1 + D)$. Widening the window (a larger t_1) only pushes ρ'_u further out; it can never make the allowance appear reset, since that needs $t_0 \geq \rho_u$ and t_0 is the earliest slot the transaction can occupy. Hence the draws funded by one reset total at most perDay_u, and two resets are always at least D apart, since every draw moves the deadline to $\rho'_u \geq t_1 + D$; together these give the rolling-window bound. □

Theorem 4 (Proof-of-life ceiling). *A* RenewProofOfLife *transition must* strictly *change the deadline, and advances it to at most one increment past the transaction's earliest time: the new deadline* d' *satisfies* $d' \neq d$, $t_1 \leq d' \leq t_0 + \Delta$ *and* $d' \geq d$ *— hence* $d' > d$.

Proof sketch. V_{S} accepts RenewProofOfLife only if the deadline actually changes *and* these three conjuncts hold; combined with $d' \geq d$ the change can only be forward. Rejecting an unchanged deadline matters for more than tidiness: the window conjuncts pass trivially when $d' = d$, so a keeper could otherwise replay a bit-identical no-op every block and contest the single state thread at fee cost — the same churn the settlement cadence bounds. The upper bound $d' \leq t_0 + \Delta$ holds regardless of t_1, so a wide window cannot skip the deadline arbitrarily far ahead. The same window check gates every transition that moves the deadline except the operator's UpdateState, which may reconfigure the timer from scratch (Section 8). □

Theorem 5 (Weighted-share recovery). *Let* $W = \sum_{b \in B} w_b$ *over the beneficiaries present in S. An unlocked beneficiary i may withdraw, per asset x, at most* $\lfloor (w_i/W)\,\mathrm{pool}_x \rfloor$, *where* $\mathrm{pool}_x = \max(0,\, \mathrm{in}_x - R_x(t_1))$, *and is removed from S in the same transition. No beneficiary withdraws twice, and the set collectively recovers the whole distributable pool in any order.*

Proof sketch. The bound is enforced division-free as $\mathrm{take}_x \cdot W \leq w_i \cdot \mathrm{pool}_x$. Because the acting weight is retired on use and W is recomputed over the beneficiaries still present, the shares of any remaining subset re-normalize to the full pool; summing over any withdrawal order telescopes to pool_x. Removing i in S' gives the one-shot property. □

Theorem 6 (Streaming reserve). *On every spend the wallet retains, of each asset x, at least the lesser of its holding and the reserve,* $\mathrm{out}_x \geq \min(\mathrm{in}_x,\, R_x(t_1))$*: streaming value accrued but unpaid as of* t_1 *cannot be moved out, and an asset already below its reserve cannot be drawn down further.*

Proof sketch. V_{W} requires $\mathrm{out}_x \geq \min(\mathrm{in}_x, R_x(t_1))$ per asset. The guard is point-in-time: it covers accrual only to t_1; Section 8 records the consequence. □

Theorem 7 (Settlement cadence)**.** *Let* $C = 1\,800\,000$ *(thirty minutes) and* $L = 3\,600\,000$ *(one hour). Every* PayStreamingPayment *must declare a finite validity interval and carry a stakeholder signature: an admin, a quorum, a listed user, the payee of some stream in* M*, or an unlocked beneficiary. Every* CancelStreamingPayment *must declare the same bounded interval and carry the target payee's signature. A payout not signed by an admin, and every payee cancellation regardless of additional signatures, must satisfy* $t_1 - t_0 \leq L$ *and* $t_0 \geq \tau + C$*, and stamps* $\tau' = t_1$*; an admin payout leaves* τ *unchanged. The true on-chain times of consecutive cadence-limited actions then lie at least* C *apart, and no such action defers the next admissible one by more than* $L + C$ *past its own earliest bound.*

Proof sketch. A transaction validates only inside its declared interval, so its true time lies in $[t_0, t_1]$. The previous cadence-limited payout or payee cancellation stamped τ with its t_1, at or after its true time; the next such action needs $t_0 \geq \tau + C$, so its true time follows the previous one's by at least C. The bound choice is what makes the gate non-gameable: gating on the *earliest* bound keeps a wide window from faking an elapsed cooldown, and stamping the *latest* bound keeps a past-dated stamp from shortening the next one. The cap $t_1 \leq t_0 + L$ bounds the stamp's overshoot, so one action cannot stamp far ahead and freeze later ones: the next is admissible by $t_0 + L + C$. An admin payout leaves τ unchanged, so it defers nothing. The stakeholder-signature requirement on payouts and target-payee signature on cancellations remove the unbounded population: a party with no relevant key cannot contest the state thread at all, at any fee. □

Theorem 8 (Recovery reachability)**.** *Every state reachable through a* configuration *transition retains an authority path: a* signable *admin record (one carrying at least one wallet key), a satisfiable multi-signature quorum, or a signable beneficiary. The sole exception is the terminal recovery draw, treated below.*

Proof sketch. Configuration validation enforces this at creation and on every UpdateState, and the linear-cost RemoveAccessIndex re-checks it. Hence an admin-less, recovery-less state is unreachable by configuration. One transition is deliberately outside the invariant: UseBeneficiary removes the acting beneficiary, so the *last* beneficiary of an operator-less wallet removes the final path as it withdraws. That is the intended terminal state of a completed recovery, not a bricking bug — the guard is not re-applied there precisely because the handler *requires* the acting beneficiary to be removed, so applying it would make the final draw impossible and strand every single-beneficiary wallet. Section 8 states the residual duty this places on the off-chain builder. Signability is enforced for all three arms uniformly: the admin arm counts an is_admin record only when it carries a wallet key, matching the multi-signature arm's positive-power-with-wallet rule and the beneficiary arm's wallet requirement. A wallet-less admin record can never satisfy the operator gate (which matches the transaction signatories against the record's wallet list), so counting it as a path would admit a state whose sole entry is such a record — permanently stranded, contradicting goal G2. Which *key* an owner uses for a signable admin, and its custody, remain the user's risk (Section 8); reachability concerns only that a satisfiable path structurally exists. □

Theorem 9 (Stake-credential pinning)**.** *Every continuing wallet output rests at the intended stake credential sc, which changes only through the dedicated operator transition.*

Proof sketch. V_W rejects any continuing wallet output whose stake credential differs from *sc*, the settlement crank included; *sc* is writable only by SetIntendedStakeCredential under admin or quorum authority. □

7 Security Analysis

Following the asset-based method of Section 3, we state the invariant defended for each asset and how the validators enforce it. Each is backed by a regression test that reproduces the corresponding attack and asserts the attack fails. Where this section argues from the protocol's assets and adversary, Section 9.2 maps the same mechanisms onto the practical loss scenarios they contain.

Single state thread.
Minting forces exactly one state token, and every spend forces exactly one continuing output carrying it; a transaction presenting a second token at the script, or none, is rejected. The State is therefore always unambiguous.

Bounded movement.
By the two-validator guarantee, outgoing value is bounded by the declared action, and the declared action is proven equal to the true state change. An owner spend is broad but still floored by the reserve; allowance, beneficiary, streaming, and consolidation spends are each bounded exactly as in Table 2.

Allowance velocity.
The daily reset is computed from the transaction's earliest validity bound, and the next deadline from the latest bound. An adversary cannot widen the validity window to fake a reset, nor anchor to a stale bound to spend twice in a day.

Proof-of-life ceiling.
A renewal's new deadline must lie at or beyond the transaction's latest bound and at most one increment past its earliest bound, so a wide validity window cannot push the deadline more than one increment ahead.

Streaming reserve, per asset.
On every spend the wallet must retain, for each asset owed to a streaming payment, at least the lesser of what it holds and what it owes for accrual up to the transaction's upper validity bound (a point-in-time reserve, not the full forward schedule; see Section 8). Per-asset accounting means a spend in one asset is never blocked by a payment in another, and a payment denominated in an asset the wallet does not hold cannot strand unrelated funds.

Payout integrity.
A streaming settlement may only increase paid-out progress, must route each asset to the configured payee address carrying the matching output tag, and, for a non-ada asset, the tagged outputs must sum to *exactly* the computed accrued delta with the STT value preserved. For an ada-denominated stream the tagged outputs must sum to *at least* the delta: because the

payout asset is ada and every output must itself carry a minimum ada amount, an exact-delta payee output below that minimum is unconstructible, so the crank tops the payee output up to a valid amount with its own ada (see Section 8, "ADA settlement granularity"). This relaxation does not widen what the wallet can lose: the wallet's *net* outflow is independently pinned to exactly the delta, and no output other than a wallet self-return, the *full-address* state-token continuation, or a correctly-tagged payee output may carry the payout asset — lovelace included. Full-address matching matters because another output may share the state validator's payment credential under a different stake credential without being the token-bearing continuation; such an output is not exempt. This closes the double-satisfaction pattern where an external input funds the payee while wallet value leaks elsewhere. What the ada case gives up is only per-payee exactness: with several ada streams settled in one transaction the wallet-sourced ada may be split across their (all configured) payees, which is value-neutral — no drain, no leak, and each payee still receives at least its settled amount. As the settlement crank is the wallet spend whose submitter need not be the owner, it is additionally bounded against UTxO-layout griefing: the number of continuing wallet outputs may not exceed the number of wallet inputs it consumes, so a cranker can consolidate or preserve the wallet's UTxO count but never fragment it into ever more dust; the owner can re-split through any authorized spend. For the same reason no continuing wallet output may carry a reference script: an attached script is charged per byte on every later transaction that consumes the UTxO and counts against the per-transaction reference-script limit, so without the ban a cranker could bloat the wallet's UTxO set until a one-shot beneficiary sweep no longer fits in a single transaction. This mirrors the identical ban on the state-token output.

Settlement cadence.
A payout must be signed by a stakeholder — an admin, a quorum, a listed user, a stream payee, or an unlocked beneficiary — and a cancellation by the target stream's payee, so an arbitrary third party cannot use either path. Every non-admin payout and every payee cancellation must further start at least thirty minutes past the previous shared stamp, inside a validity window of at most one hour. The limit is gated on the transaction's *earliest* bound and stamped from its *latest*, so a wide window cannot fake an elapsed interval and a past-dated stamp cannot shorten the next one. A payee may shorten the same end again after the cooldown; this bounded, authorized churn is accepted.

Recovery reachability.
At creation and on every state update, the configuration must retain a reachable access path: a signable admin (an admin record carrying at least one wallet key), a satisfiable multi-signature quorum, or a signable beneficiary (which, under weighted shares, can sweep the whole pool). A wallet-less admin can never sign, so it is not counted as a path — the same signability standard the other two arms already apply; a state whose only entry is a wallet-less admin is therefore rejected rather than accepted as stranded. An admin-less, recovery-less dead-end is unrepresentable (goal G2).

Credential aggregation.
Wallet value is aggregated by payment credential, not full address, so a beneficiary cannot multiply its share by spreading wallet outputs across stake-credential variants of the same script in one transaction.

Stake-credential pinning.
Every continuing wallet output must carry the State's *intended stake credential* (Section 4.5). No spend, the settlement crank included, can re-home wallet funds to a foreign ("Frankenstein") stake credential to capture their staking rewards, delegation, or governance vote, or to hide them from address-based discovery; only an admin or quorum may change the intended credential, through a dedicated transition.

Bounded execution cost.
Each access list has a cap (at most 15 user records, with owners, spenders, and liveness keepers sharing this budget; 25 beneficiaries; and 25 streaming payments), so that validating and transitioning the State always fits a transaction's execution budget. Configuration growth runs only through the full-validation update path, by construction the most expensive transition. An addition that would approach the budget ceiling therefore fails *before* any cheaper spend or recovery would, so growth can never leave the wallet unable to spend or recover. A dedicated linear-cost removal, which skips the full validation, lets an over-large configuration always be shrunk back, one entry at a time.

The authorization-gate stance (G3) is a deliberate trade-off, taken up in Section 8: a quorum of owner keys can move funds freely down to the reserve, because the design's safety rests on the authorization gate and dead-man-switch timing rather than on on-chain spend restrictions.

8 Limitations and Trust Assumptions

This section lists what the design does not protect against and the trust assumptions it makes.

Trust assumption

The dead-man-switch protects against the owner going *silent*, not against a compromised key while the owner is active: anyone holding sufficient owner authority can spend and reset the timer. Liveness also depends on the off-chain wallet renewing proof of life on the owner's behalf. The on-chain timer enforces the deadline, but choosing a sensible timeframe and trustworthy beneficiaries remains the owner's responsibility.

Broad operator spend (by design, G3).
A quorum of owner keys can drain the wallet down to the streaming reserve. The mitigations are the dead-man-switch, the recovery-reachability invariant, multi-signature thresholds, and trusted transaction construction, not on-chain destination or amount limits. Narrowing the operator envelope would be a product change, not a security fix.

Multi-signature counts power per record, not per key.
Signing power is summed per user record, and the contract enforces unique record *ids*, not unique keys. A key placed in several multi-signature records therefore contributes its power once for each, so a threshold that looks like it needs several distinct signers can be satisfied by fewer. Surfacing duplicate keys across records is the off-chain configuration interface's responsibility; read a threshold as a sum of record weights, not a count of distinct people.

Admin-only wallets are permitted.
A wallet may be configured with a sole admin and no other recovery path; losing that key is then an accepted user risk, exactly as for any single-signer wallet. The reachability invariant only guarantees that an *admin-less* wallet always retains a non-admin way back in.

Shape, not timing, of recovery.
The contract rejects only genuinely bricked configurations; a beneficiary whose unlock lies far in the future is accepted on-chain and flagged off-chain. A trusted liveness keeper can defer beneficiary unlock indefinitely while it keeps renewing, and an operator (admin or quorum) can push the deadline far forward in a single reconfiguration: the one-increment cap binds only the heartbeat renewal paths, not the full state-update path, which must be able to (re)configure the timer from scratch.

Streaming payments are additive and end-date governed.
An operator (admin or quorum) may add streaming payments to a live wallet as well as configure them at mint; each addition is born unsettled, and the set stays bounded by the count cap. An existing payment can never be deleted through management or have its immutable fields changed, and its end date can only move down to (never below) the transaction's upper validity bound, so already-accrued value is never clawed back; it may also be extended. A payment's own payee may shorten its stream through a signature-authorized cancel under the same no-clawback principle. Before the start it may cap exactly to the start, yielding a zero-lifetime obligation rather than forced millisecond accrual. There is no cancellation marker: after the shared cooldown the payee may shorten it again, and an operator may later reschedule it. A payment leaves the set only through settlement. The reserve accounting stays sound because it is recomputed per asset from the live payee set on every spend, not assumed over a fixed set.

Streaming reserve is point-in-time.
The reserve protects only streaming-payment value *accrued* as of a spend's upper validity bound, not the full remaining schedule. A beneficiary recovering after the owner goes silent can therefore draw down principal that a payment's *future* accrual will need, leaving later settlements underfunded. Reserving the whole forward schedule would let long-dated payments starve heirs, so the design protects only accrued value; keeping enough on hand for future accrual remains the owner's planning responsibility.

Permanent state thread.
There is no burn/close path; the small state-token deposit persists even after funds are fully recovered. This keeps the "exactly one STT, always" invariant unconditional.

Open settlement timing.
Any stakeholder — an operator, a listed user, a stream payee, or an unlocked beneficiary — may trigger a streaming payout (and pay its fee) at a moment the owner might have deferred. No value can be misdirected, since funds reach only the configured payee, in exactly the amount the settlement records and never more than is owed, but settlement timing is not the owner's to control: the cadence limit bounds how *often* other stakeholders may settle, not *when* they choose to. Parties with no relationship to the wallet cannot settle at all.

Under-funded settlement is first-come, not pro-rata.
The reserve gate is a floor on discretionary outflow, and settlement is exempt from it: a settlement's outflow is independently pinned, asset by asset, to the tagged payee outputs, so it cannot

underfund anyone and needs no floor. Applying the floor to settlement was in fact a deadlock — when a wallet holds less of an asset than it owes, the floor forbids *every* outflow of that asset, including the only action that can reduce the debt, so the asset would freeze permanently for payee, operator and beneficiary alike while accrual kept growing. The exemption removes the freeze at the cost of ordering: when a wallet cannot cover everything it owes, payees are settled in the order they crank rather than proportionally. Each payment is still individually capped at its own accrual, so no payee is ever paid more than it earned; keeping the wallet funded above its accrued obligations remains the owner's planning responsibility.

Terminal recovery state.
A wallet whose only authority path is its beneficiaries ends, once the last beneficiary draws, in a state with no access path at all: the state token is permanent, so the wallet address stays a valid deposit target that nobody can ever spend from. This is the intended end of a completed recovery rather than a defect — the alternative, refusing the final draw, would strand every single-beneficiary wallet instead. The contract enforces only the *upper* weighted-share bound and no minimum withdrawal, so the recovering beneficiary must sweep every asset in the same transaction, and must not treat the address as live afterwards. Both duties belong to the off-chain builder, which is responsible for constructing the full sweep and warning the user.

State-token value is a one-way ratchet without an admin.
The state token's own output may always receive more ada, but only an admin settlement-free *Use* may reduce it. In a wallet with no admin — quorum-only or beneficiary-only — any ada pushed into that output is therefore locked for the wallet's lifetime, which is permanent. The amounts involved are min-UTxO tier and no path lets a third party push value there without the owner's own authorization, so this is a construction footgun for the off-chain builder rather than an attack surface.

Only token-bearing outputs at the state-token address are spendable.
Every state token under the policy shares one address. A UTxO sent there *without* the token can never be spent: alone it fails the single-token check, and alongside the real state token it fails the "exactly one input at this address" filter. Such a UTxO can only be created by whoever funds it, so this is self-inflicted rather than a griefing vector, but any off-chain component that indexes the state-token address must filter by token, and a mis-built transaction that pays the address silently burns its funds.

ADA settlement granularity and fee funding.
The payout-integrity routing (Section 7) forbids *any* non-protocol output from carrying the payout asset, and every Cardano output must carry a minimum amount of ADA. For an ADA-denominated stream the payout asset *is* ADA, with two consequences. First, a settlement funded from wallet value cannot emit the cranker's own fee-change output — that untagged change output would carry the payout asset and be rejected — so the cranker's funding input has only two legal ADA sinks, the tagged payee output and the transaction fee, and is allocated across the two with no residual returned (the fee is *not* equal to the input once a top-up is present: part of the input tops the payee output up to the min-UTxO amount described next, and the remainder pays the fee). Second, the settled amount can be smaller than the per-output ADA minimum, and an exact-delta payee output would then be unconstructible. To keep small ADA settlements possible, the tagged-output check for ADA requires the payee outputs to sum to *at least* the delta rather than exactly it (Section 7, "Payout integrity"): the cranker tops the

payee's own tagged output up to a valid amount with its own ADA. The anti-leak guarantee stays airtight — the wallet's net outflow is still pinned to exactly the delta and can reach only a configured payee — while the surplus is the cranker's, not the wallet's. The residual cost is only per-payee exactness across multiple simultaneous ADA streams, which is value-neutral (see "Payout integrity"). Constructing the crank (no change output; sufficient top-up on small ADA settlements) is the off-chain builder's responsibility. Streams denominated in a non-ADA asset are unaffected: their min-UTxO padding is in ADA, a different asset, so their tagged outputs remain pinned to the exact delta.

9 Illustrative Use Cases

The model composes into concrete configurations. The following are illustrative, not exhaustive.

9.1 Classical use cases

Inheritance and backup. An owner keeps unrestricted control and names one or more beneficiaries behind the dead-man-switch. While the owner transacts (or sends proof of life), beneficiaries can do nothing; if the owner goes silent past the deadline, beneficiaries recover their weighted shares. This transfers assets on death or key loss without ever sharing a seed phrase, and beneficiaries can be changed at any time, for example after marriage. Trust in beneficiaries is bounded (each takes only its share) and revocable.

Inheritance with vesting. As above, but a beneficiary also cannot unlock before a chosen date, for instance before reaching legal adulthood. This limits premature access by guardians while the owner's proof of life still gates recovery.

Shared treasury with allowances. A family or small team pools funds in one wallet: large moves require a multi-signature quorum; members hold capped daily allowances for everyday spending; salaries or vendor retainers run as streaming payments that settle automatically without anyone online; and a trusted party is named beneficiary so the treasury stays recoverable if the active signers are lost.

9.2 Threats mitigated in practice

The mechanisms of Sections 5 and 7 address two families of risk: the catastrophic loss or seizure of funds, and the everyday risks of managing them. Table 3 covers the first family, listing common ways funds are lost or seized, the configuration that contains each, and the most an adversary gains once that configuration is in place. Table 4 covers the second. Neither list is exhaustive, and every entry inherits the trust assumptions of Section 8. In particular, the authorization-gate stance (G3) means these protections come from *distributing and capping authority*, not from restricting where an authorized key may send funds.

Beyond catastrophic loss, the same vocabulary addresses the everyday risks of managing funds: spending one's own holdings too freely, a slip of the hand, a convincing scam, or a shared pool any one person can move alone. Table 4 lists these, where shared control through weighted multi-signature does most of the work.

One scenario is worth spelling out, since its bound is the one users most often misread.

Physical coercion (the "wrench attack").
When the person under duress holds only a spender key, the contract refuses to release more than that key's remaining daily allowance, whatever the holder is forced to sign. Reaching the bulk of the funds needs admin or multi-signature authority the coerced party does not carry, and under a quorum no single individual present can satisfy it. Because the cap is per day rather than per transaction, the bound holds even under sustained force: a captor can extract at most one daily allowance for each day the victim is held, never the bulk. Forcing more therefore means holding the victim longer for a small marginal sum, which raises the captor's exposure faster than the payout. The effect is that of a low-balance travel wallet, but enforced on one shared pool rather than by splitting funds across separate wallets. The limit (G3, Section 8) is that coercing a *sole admin*, or a whole quorum at once, is not contained on-chain; safety here comes from not carrying full authority on the key used to travel or transact day to day.

Transition	Authority
Operator use	admin or multi-signature
Update state	admin or multi-signature
Remove access entry	admin or multi-signature
Manage streaming payments	admin or multi-signature
Set stake credential	admin or multi-signature
Renew proof of life	a liveness keeper
Use allowance	the spending user
Use beneficiary	one unlocked beneficiary

Threat	Containing configuration	Most the adversary gains
Physical coercion or kidnapping (the "$5 wrench") on an everyday key	Day-to-day use on a capped spender; owner authority held separately	One day's allowance per day held, never the bulk
Single key stolen (malware, phishing, lost device)	Weighted multi-signature over owners; everyday use on a spender	Nothing, or one day's allowance, until a quorum is also taken
Malicious dApp or wallet-drainer signature	Interaction through a low-allowance spender, not an owner key	One day's remaining allowance
Rogue insider in a shared treasury	Large moves need a quorum; members hold capped allowances	One member's daily allowance
Sudden death or incapacitation	Proof-of-life dead-man-switch with named beneficiaries	Nothing; beneficiaries recover only after the deadline
Lost seed or hardware failure	A backup key named as one's own beneficiary	Nothing; recover to the backup after the deadline
Greedy or dishonest heir or executor	Weighted-share recovery, one withdrawal each	Only that heir's weighted share
Premature access by a guardian	Beneficiary vesting delay (e.g. until adulthood)	Nothing before the vest date
Dependent left without income	Streaming payment behind a protected reserve	Cannot divert or underfund already-accrued payouts
Staking-reward or governance-vote capture via a foreign stake credential	Stake-credential pinning and the sweep diagnostic (Section 11)	Nothing; rewards, delegation, and vote stay with the wallet
Griefing the configuration to brick the wallet	Capped access lists, linear-cost removal, reachability invariant	Cannot strand funds or block recovery

Table 3: Catastrophic loss-and-seizure scenarios and the configuration that contains each.

Risk	Containing configuration	What the configuration ensures
Impulsive or emotional moves (panic-sell, FOMO, gambling)	A daily allowance set as one's own cooling-off cap	At most one day's allowance leaves per day; a larger move takes a deliberate second step
Operational mistake on an everyday key (wrong amount or address)	Everyday use on a capped spender	A single slip cannot exceed the daily cap
Scam or social engineering of one signer	Weighted multi-signature, a second signer reviewing independently	No move until a quorum approves on its own devices, so one deceived signer is not enough
Malware or a compromised device on one signer	Multi-signature approvals gathered on independent devices	One compromised machine cannot move funds alone
Unilateral or unaccountable move of shared funds	A multi-signature quorum over treasury actions	No single member moves funds alone, and each approval leaves a shared on-chain record
A signer temporarily unavailable (travel, illness, lost device)	An m-of-n threshold with $m < n$	The wallet keeps operating with the remaining signers; total loss is still covered by the dead-man-switch

Table 4: Everyday control and human-factor risks, and the configuration that contains each.

10 Related Work

Per-day allowance with counterparty recovery. The closest published analog is the savings-wallet sketch in the Ethereum whitepaper [4]: the owner may withdraw at most 1% per day, a bank counterparty the same, the owner can shut off the bank's power, and together they may withdraw anything. This is prior art for the per-day-allowance-plus-single-counterparty pattern. It has no weighted-share or multi-beneficiary recovery, no inheritance, and no dead-man-switch; that is the space this design occupies.

Smart accounts and social recovery. Account-model smart-account standards such as ERC-4337 [12], and wallets built on them, add programmable authorization and guardian-based social recovery. They target a mutable-storage account model. Our contribution is a comparable permission-and-recovery surface expressed natively in EUTXO, where state is threaded through a token rather than mutated in place (Section 2).

On-chain multi-signature. Multi-signature wallets (for example Safe [13] on EVM chains, and Cardano's native and Plutus multi-signature) distribute trust across signers. They are a building block here, since weighted multi-signature is one authority path, but on their own they lack time-based recovery: there is no notion of owner inactivity triggering a fallback.

MPC and threshold-signature custody. MPC and threshold-signature wallets split a key across parties off-chain. They are sometimes presented as a uniformly safer default, but the evidence does not support that: zero-day vulnerabilities were found across more than fifteen production MPC/TSS implementations [14]. We therefore position an on-chain, validator-enforced model not as cryptographically superior but as differently trusted. Its rules are public, deterministic, and open to review (G4), with no off-chain signing protocol to get wrong.

11 Implementation

On-chain validators. The validators are written in Aiken [15], chosen for its developer experience, built-in testing, and fit for an open-source contract meant to be read and reviewed (G4). AI assists development, widening test coverage: it generates unit and regression tests, drafts the attack scenarios behind each invariant of Section 7, and expands property-based fuzzing inputs that probe boundary conditions a hand-written suite tends to miss. Every generated test is reviewed before it enters the suite, and the security invariants it checks are stated by us, not inferred by the model. On-chain enforcement is split as in Section 4: the wallet validator, the one holding the funds, compiles to under half the size of the STT validator, and the always-fail reference store to under a hundred bytes. The Aiken suite spans unit checks, an attack-regression log, and property-based fuzzing.

Off-chain stack and interface. Everything else (building, balancing, and submitting transactions) happens off-chain, where we use Mesh [16]. A reference web interface, built with Next.js [17], connects to any CIP-30 browser wallet [18] (and, over WalletConnect [19], to mobile or remote wallets), previews every transaction before signing, and routes chain queries through a server-side proxy so no third-party API key reaches the browser. Day-to-day actions are presented as guided flows, with staking, governance, and recovery actions on an advanced surface.

Stake-credential diagnostic. One diagnostic is a stake-credential check: it queries the wallet's *payment* credential against a public indexer, so it finds funds resting at an unintended ("Frankenstein") or outdated stake credential that an address-based query would miss, and offers to sweep them back to the intended address. The indexer's public API sends no cross-origin access header, so the browser cannot query it directly; the call goes through a small same-origin proxy on the application server, which forwards it unchanged. The trade-off is that the server sees which payment credential is being checked; we accept it because the query does not work from the browser otherwise. The check runs once when a wallet is opened and on demand from the tools surface. Figure 6 sketches the end-to-end interaction.

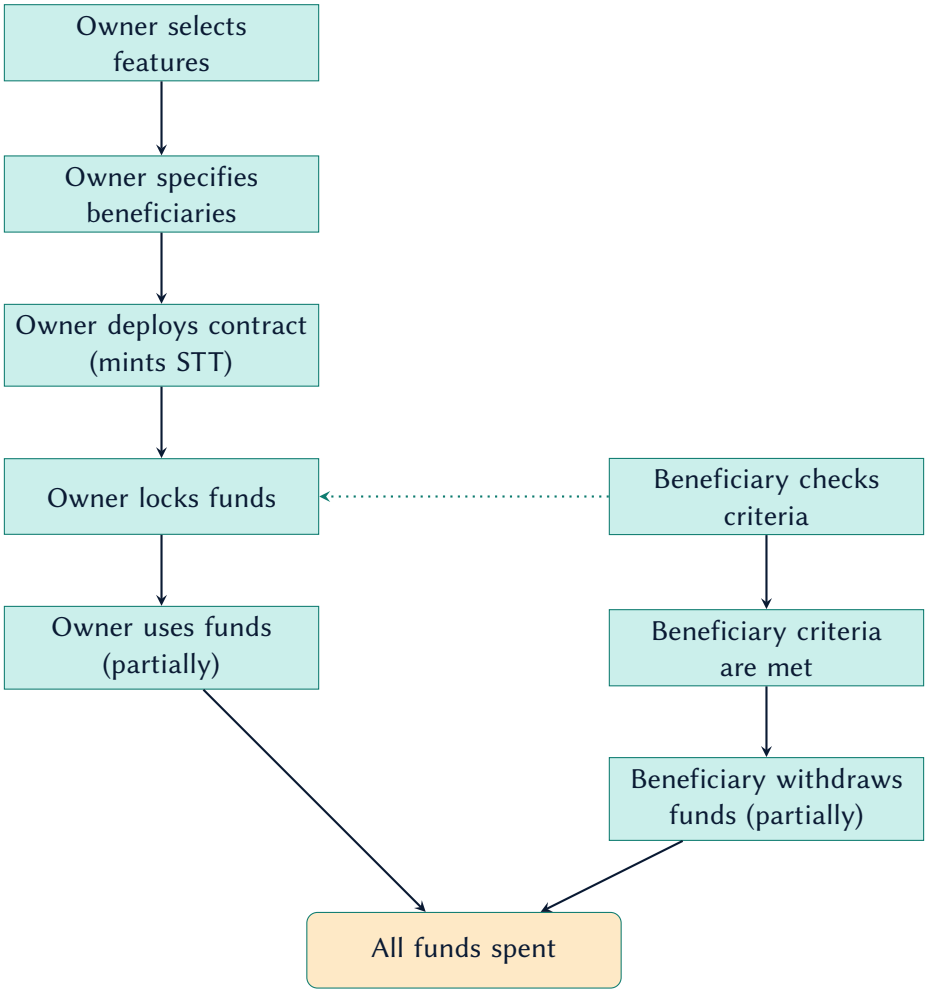

Figure 6: End-to-end interaction, from wallet creation to recovery.

12 Conclusion

This design threads a shared wallet's entire configuration through a single state-thread token and bounds every movement of funds with a two-validator handshake. Spending limits, weighted multi-signature, a proof-of-life dead-man-switch, weighted-share multi-beneficiary recovery, and stakeholder-settled streaming payments are then *configuration* on one reviewable EUTXO contract, while custody stays with the owner throughout. The security argument is small enough to review in full: one composed guarantee (Section 4), an invariant defended per asset (Section 7), and the limits and trust assumptions of Section 8. The contracts and the reference interface are open source.

Status and future work. The implementation currently targets the Cardano Preprod test network, and the validators have not had an external audit, so the wallet should not yet hold real funds. The remaining work is tracked publicly through the project's Catalyst milestones [2].

Acknowledgments

This work was funded by Project Catalyst [1] under Fund 11 (Cardano Use Cases: Concept). We thank the Cardano community for supporting it.

References

[1] Project Catalyst, "(Dead-man-switch) Permission based Wallet — Fund 11, Cardano Use Cases: Concept," https://projectcatalyst.io/funds/11/cardano-use-cases-concept/dead-man-switch-permission-based-wallet, 2024.

[2] ——, "Milestone module — project 1100002," https://milestones.projectcatalyst.io/projects/1100002/milestones, 2024.

[3] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, "The Extended UTXO Model," in *Financial Cryptography and Data Security Workshops*, 2020, https://iohk.io/en/research/library/papers/the-extended-utxo-model/.

[4] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014, https://ethereum.org/en/whitepaper/.

[5] National Cyber Security Centre, "Protocol Design Principles," https://www.ncsc.gov.uk/paper/protocol-design-principles, 2020.

[6] G. Almashaqbeh, A. Bishop, and J. Cappos, "ABC: A Cryptocurrency-Focused Threat Modeling Framework," in *IEEE INFOCOM Workshops (CryBlock)*, 2019, https://arxiv.org/abs/1903.03422.

[7] Cardano Developer Portal, "Native Assets and Tokens," https://developers.cardano.org/docs/learn/core-concepts/assets/, 2024.

[8] Aiken, "Common Design Patterns — State Thread Token and Multivalidators," https://aiken-lang.org/fundamentals/common-design-patterns, 2024.

[9] Plutonomicon contributors, "The State Thread Token (STT) Pattern," https://plutonomicon.github.io/plutonomicon/statethread, 2022.

[10] Cardano Developer Portal, "The Extended UTxO (EUTXO) Model," https://developers.cardano.org/docs/get-started/technical-concepts/eutxo/, 2024.

[11] Cardano Foundation, "CIP-0019: Cardano Addresses," https://cips.cardano.org/cip/CIP-19, 2024.

[12] V. Buterin, Y. Weiss, D. Tirosh *et al.*, "ERC-4337: Account Abstraction Using Alt Mempool," https://eips.ethereum.org/EIPS/eip-4337, 2021.

[13] Safe Ecosystem Foundation, "Safe: Smart Account Infrastructure," https://safe.global/, 2024.

[14] Fireblocks Cryptography Research, "BitForge: Zero-Day Vulnerabilities in Multiple MPC/TSS Wallet Implementations," https://www.fireblocks.com/blog/bitforge-fireblocks-researchers-uncover-vulnerabilities-in-over-15-major-wallet-providers, 2023.

[15] Aiken, "Aiken — a modern smart contract platform for Cardano," https://aiken-lang.org/, 2024.

[16] Mesh, "Mesh — an open-source library for building Web3 apps on Cardano," https://meshjs.dev/, 2024.

[17] Vercel, "Next.js — the React framework for the web," https://nextjs.org/, 2024.

[18] Cardano Foundation, "CIP-0030: Cardano dApp-Wallet Web Bridge," https://cips.cardano.org/cip/CIP-30, 2021.

[19] WalletConnect, "WalletConnect v2.0 Protocol Specifications," https://specs.walletconnect.com/2.0, 2024.